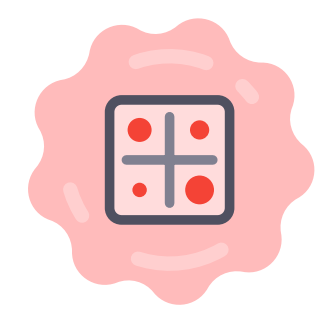
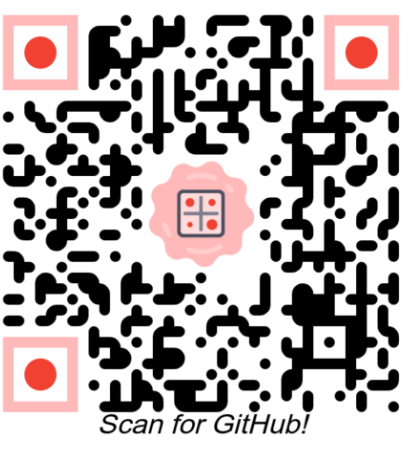




Exploring single-cell *images and profiles* together with **CytoDataFrame**

Dave Bunten¹, Jenna Tomkinson¹, Gregory P. Way¹

¹Department of Biomedical Informatics, University of Colorado Anschutz Medical Campus



Anschutz

I. Why separate profiles and images?



Figure 1: Feature tables and images often live in separate tools, making it hard to build intuition about single cells. **CytoDataFrame** keeps them together in one interactive table inside your notebook.

High-content screens produce millions of rows of features and many images. Jumping between a DataFrame and an image viewer slows exploration and hides issues like segmentation or image errors. **CytoDataFrame** puts an **image thumbnail** right next to the row's features so you can see how features relate to images.

II. CytoDataFrame for integrated analysis

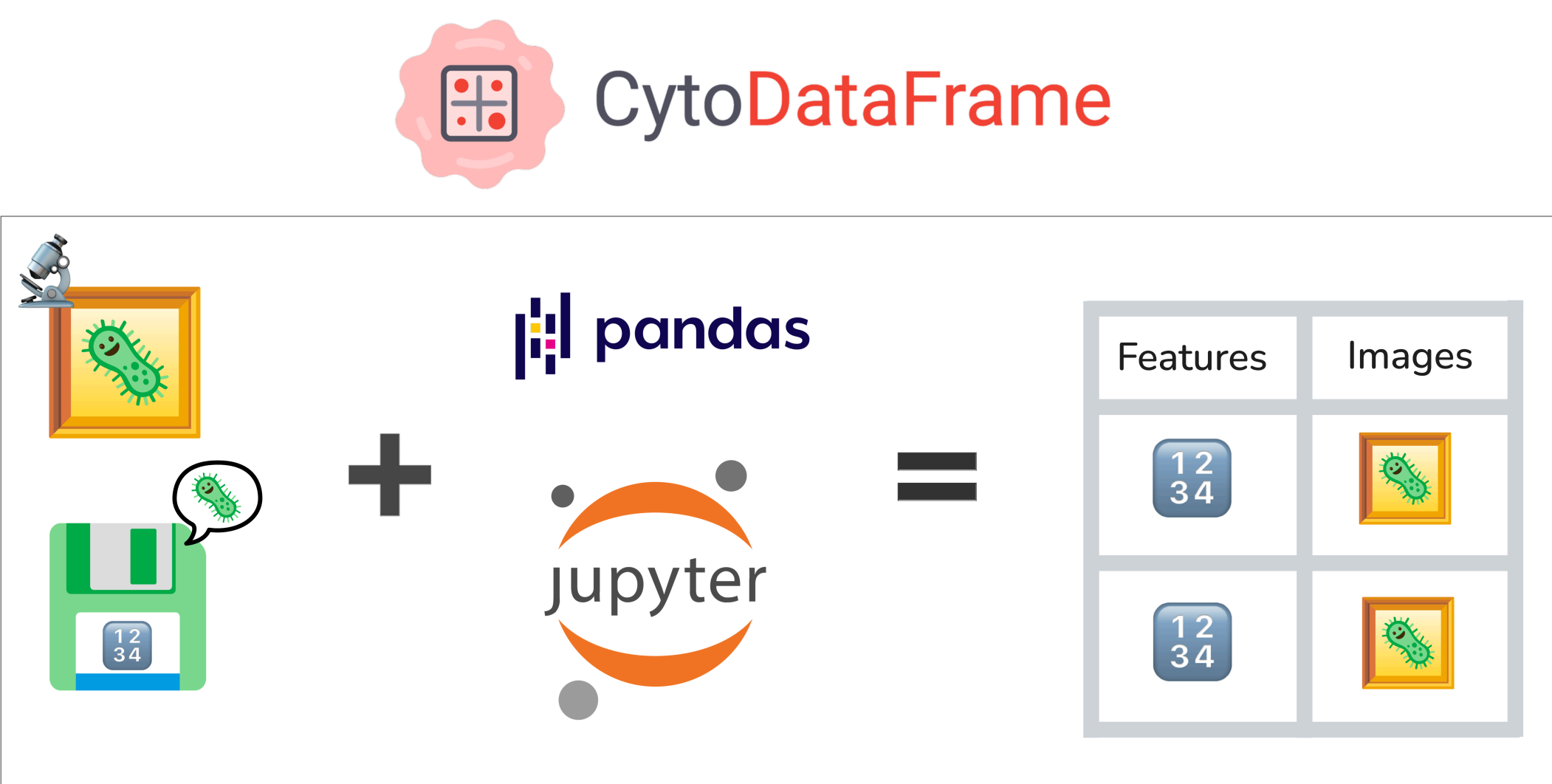


Figure 2: Images and profile data may be mixed using Pandas idioms through Jupyter interfaces to create a unified experience for bioimage analysts and data scientists.

CytoDataFrame is a small package built on top of pandas. DataFrame that embeds **inline images** (cells, nuclei, or other compartments) alongside per-object features within **Jupyter** notebook interfaces. Using **CytoDataFrame** enables you to:

- **Keep context at a glance** - Each row shows the features and the object.
- **Filter and explore** - Subset outliers, phenotypes, or QC flags and see what they look like immediately.
- **Stay in your workflow** - It behaves like a DataFrame; your plotting, modeling, and exports still work.
- **Fantastic for integrations** - Tight integration with packages such as **coSMicQC** returns **CytoDataFrames** so you can act on QC signals with visuals.

III. Using CytoDataFrame in Python

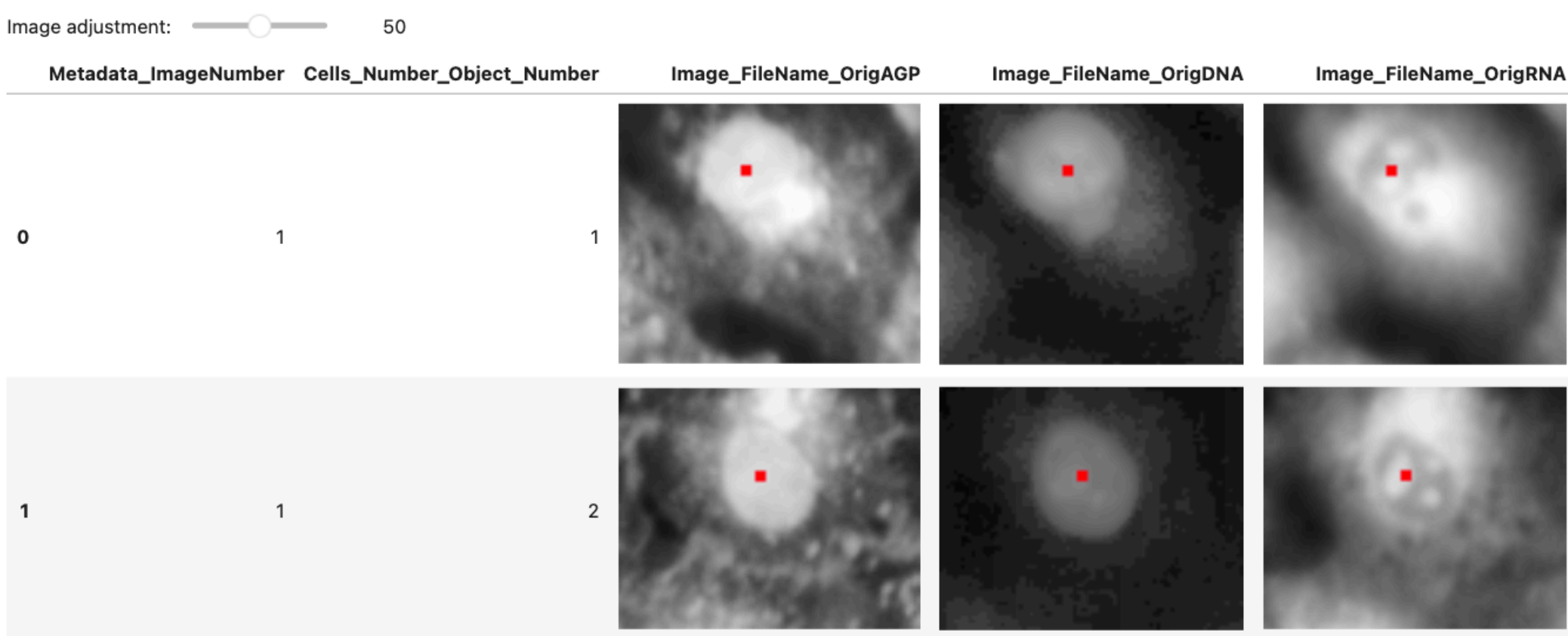
a) Install

```
# install CytoDataFrame from PyPI
pip install cytodataframe
```

b) View images with your profiles

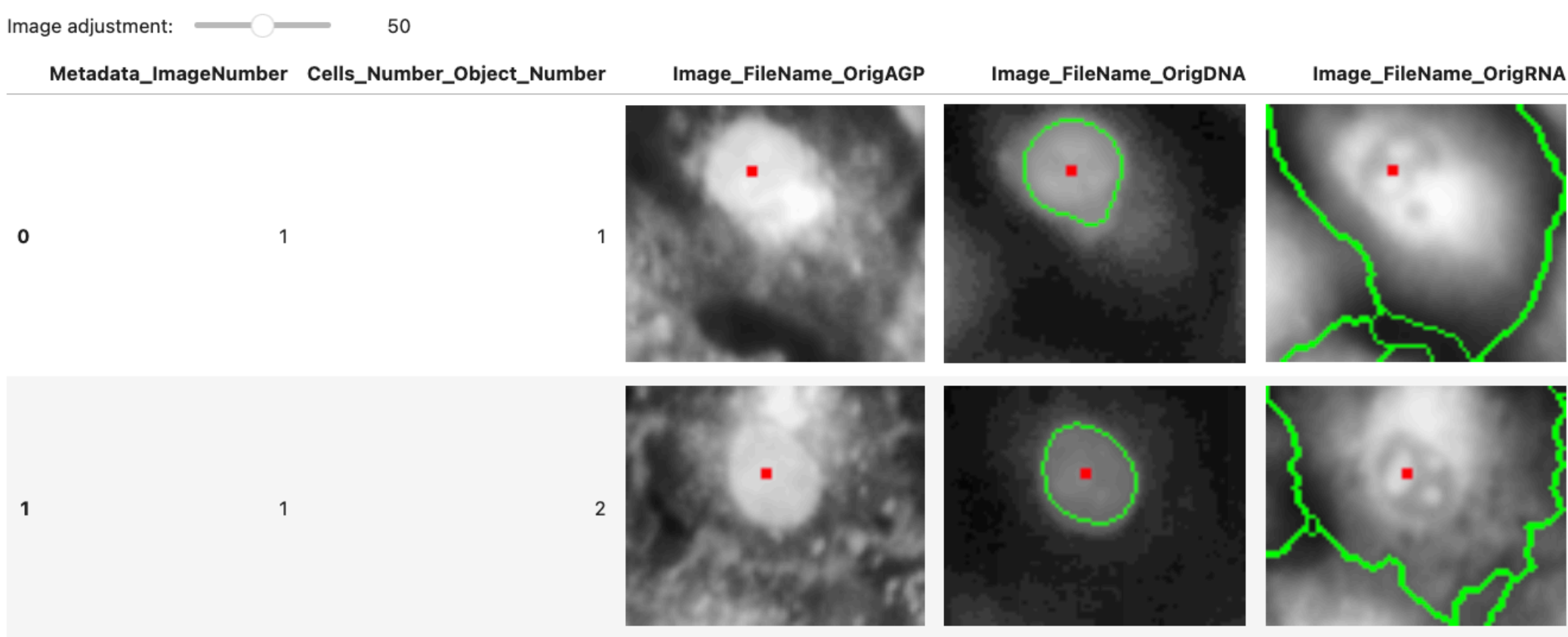
```
from cytodataframe import CytoDataFrame

# Load features and images together.
CytoDataFrame(
    data="BR00117006.parquet",
    data_context_dir="images/orig",
)[[
    "Metadata_ImageNumber",
    "Cells_Number_Object_Number",
    "Image_FileName_OrigAGP",
    "Image_FileName_OrigDNA",
    "Image_FileName_OrigRNA",
]]
```



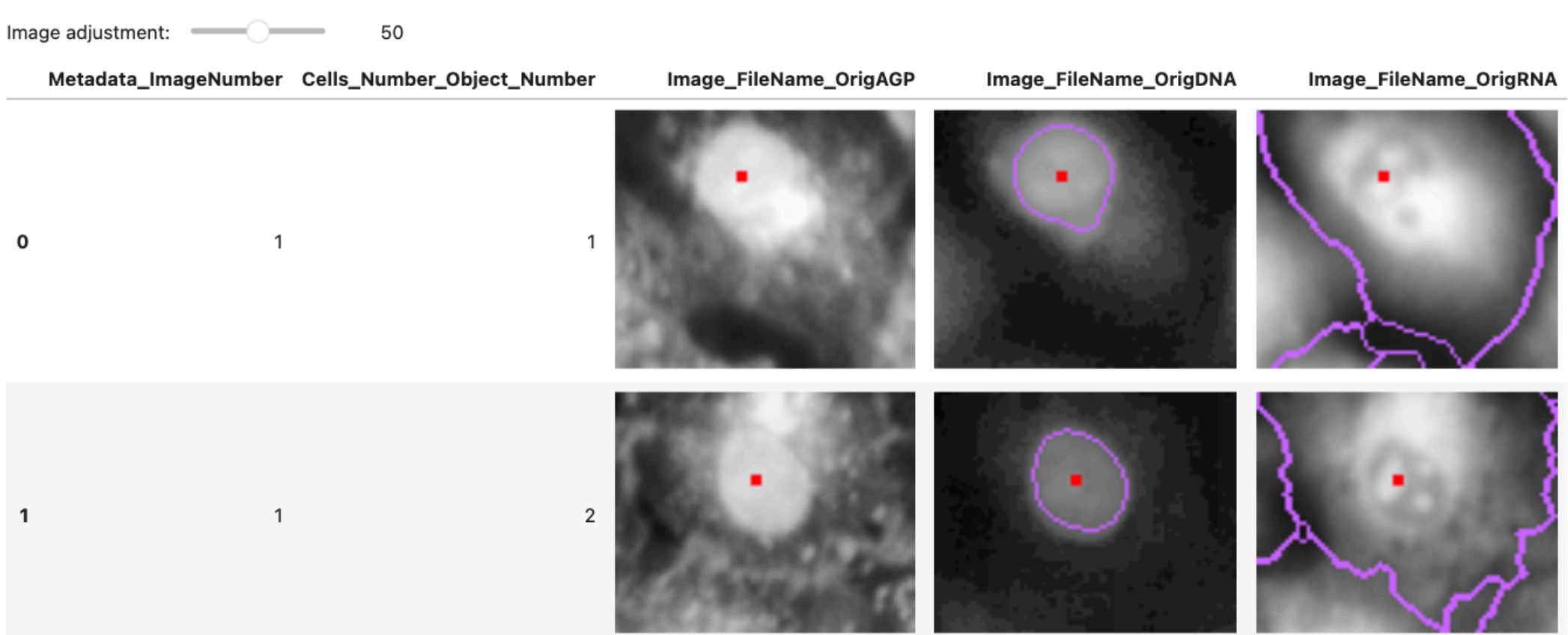
c) Show segmented objects with profiles

```
# load outlines to show segmented images
CytoDataFrame(
    data="BR00117006.parquet",
    data_context_dir="images/orig",
    data_outline_context_dir=f"images/outlines",
)[[
    "Metadata_ImageNumber",
    "Cells_Number_Object_Number",
    "Image_FileName_OrigAGP",
    "Image_FileName_OrigDNA",
    "Image_FileName_OrigRNA",
]]
```



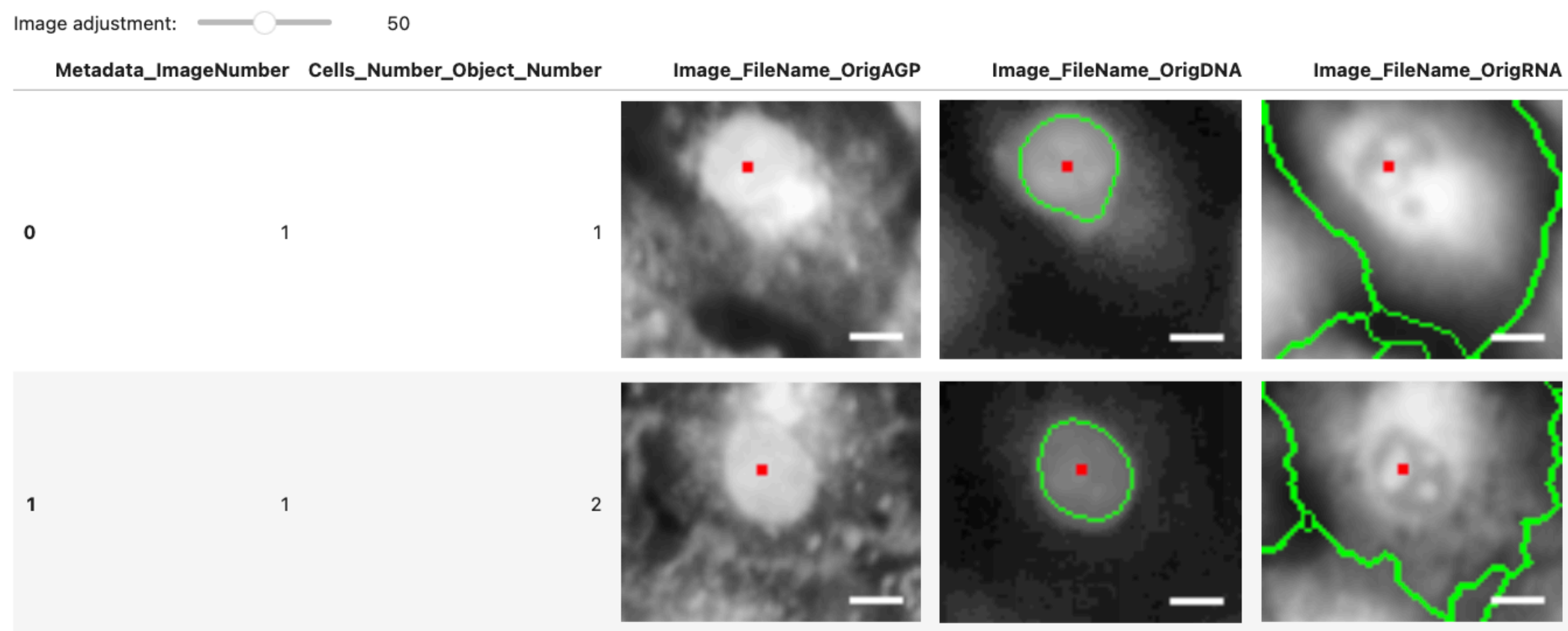
d) Change display configuration for images

```
# change the color of your outlines (and more!)
CytoDataFrame(
    data="BR00117006.parquet",
    data_context_dir="images/orig",
    data_outline_context_dir=f"images/outlines",
    display_options={
        "outline_color": (200, 100, 255)
    },
)[[
    "Metadata_ImageNumber",
    "Cells_Number_Object_Number",
    "Image_FileName_OrigAGP",
    "Image_FileName_OrigDNA",
    "Image_FileName_OrigRNA",
]]
```

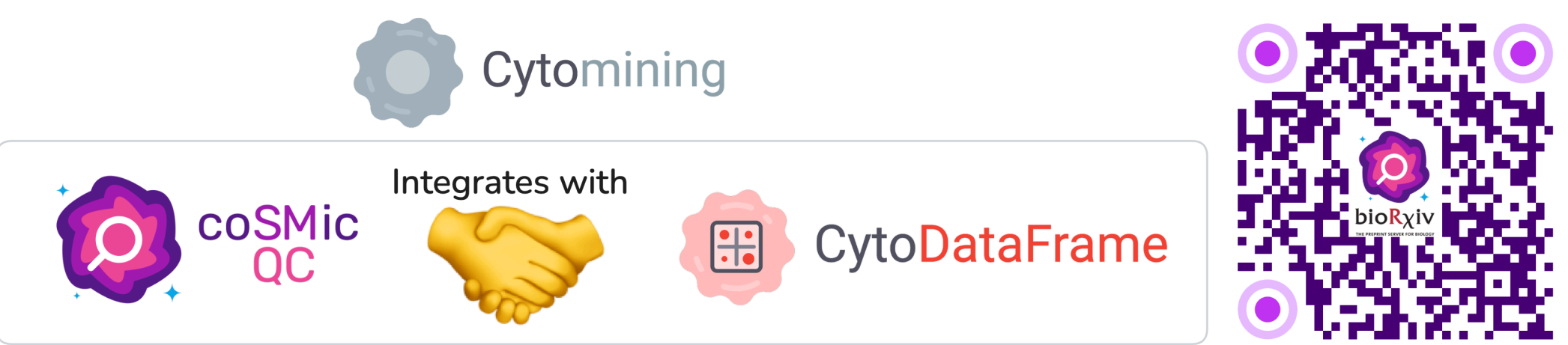


e) Add scale bars for objects

```
# add scale bars using reproducible configuration
CytoDataFrame(
    data="BR00117006.parquet",
    data_context_dir="images/orig",
    data_outline_context_dir=f"images/outlines",
    display_options={
        "pixel_per_um": 0.1550,
        "scale_bar": {"length_um": 100,
            "location": "lower right",
            "color": (255, 255, 255),
            "thickness_px": 2, "margin_px": 5,},
    },
)[[
    "Metadata_ImageNumber",
    "Cells_Number_Object_Number",
    "Image_FileName_OrigAGP",
    "Image_FileName_OrigDNA",
    "Image_FileName_OrigRNA",
]]
```

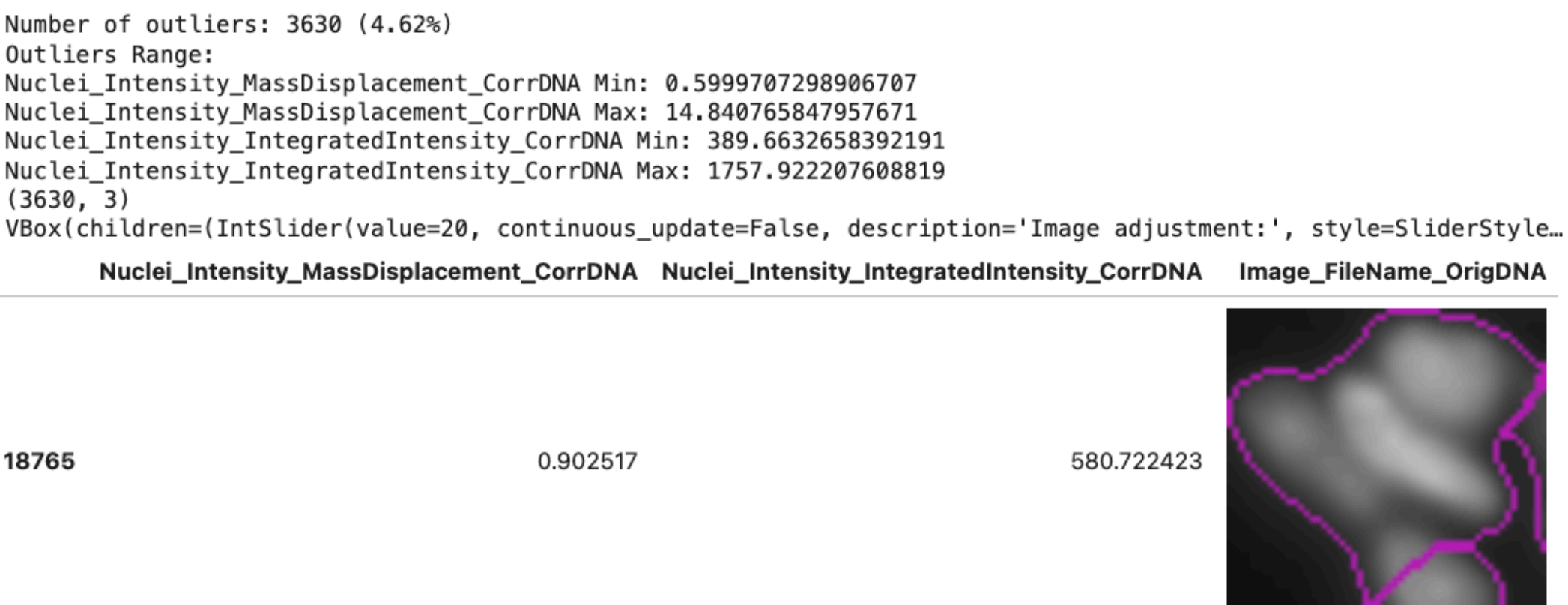


IV. coSMicQC leverages CytoDataFrames



```
import cosmicqc

# find and label outliers for single-cell data
find_outliers(
    df=CytoDataFrame(
        data="single-cell-profiles.parquet",
        data_context_dir="images/orig",
        data_outline_context_dir=f"images/outlines",
        display_options={"center_dot": False,
            "outline_color": (180, 30, 180), "brightness": 20,
        },
    ),
    metadata_columns=metadata_columns,
    feature_thresholds={
        "Nuclei_Intensity_MassDisplacement_CorrDNA": 0.05,
        "Nuclei_Intensity_IntegratedIntensity_CorrDNA": 1.5,
    },
)[[
    "Nuclei_Intensity_MassDisplacement_CorrDNA",
    "Nuclei_Intensity_IntegratedIntensity_CorrDNA",
    "Image_FileName_OrigDNA",
]]
```



coSMicQC functions can return **CytoDataFrames**, letting you pair quality control flags with images to make immediate, visual decisions and improve your profiling outcomes.

V. Acknowledgements

We thank those who have inspired, contributed, or helped support **CytoDataFrame** and the **Cytomining Ecosystem**:

- Open source science from the teams behind **CellProfiler**
- Members of the **Way Lab** at the University of Colorado Anschutz Medical Campus
- **Department of Biomedical Informatics** within the School of Medicine at the University of Colorado Anschutz Medical Campus